

EEA-EV008 - Visual Perception and Planning for Autonomous Driving

Projects Description

1 Overview

The objective of these projects is to develop and evaluate perception algorithms for autonomous driving. Each team will work with real-world sensor data to design systems that enable a vehicle to perceive and interpret its surroundings from visual inputs.

In modern autonomous driving systems, perception is composed of several specialized tasks that collectively allow a vehicle to understand complex and dynamic environments. In this project, each team will focus on **one specific perception task** and develop a complete solution for that task. This includes designing the approach, training the model, evaluating performance using appropriate metrics, and analyzing the results.

The project tasks are based on the [KITTI Vision Benchmark Suite](#), and you are encouraged to use this dataset for the development and evaluation of your algorithms. For model training, you may optionally incorporate additional datasets (for example, [nuScenes Dataset](#)) to improve model performance or robustness. However, the use of external datasets is not required.

The available project tasks include:

1. Visual Odometry / SLAM
2. 2D Object Detection
3. Road and Lane Detection
4. Segmenting and Tracking Every Pixel

By the end of the project, students are expected to demonstrate a solid understanding of the perception task they selected, implement a functioning solution, and present their methodology, experiments, and findings through a technical report and a final presentation. The project also emphasizes effective teamwork and fair contribution among team members.

2 Deliverables

For each project, each team is required to submit the following:

1. Technical report written in IEEE format. You can find the template here [IEEE Template](#)
Each report includes the following
 - (a) Abstract
 - (b) Introduction
 - (c) Methodology
 - (d) Experiments and Results
 - (e) Analysis
 - (f) Conclusion

- (g) Appendix: include the difficulties/problem you have faced when doing the project and how do you solve them.
- 2. GitHub repository of your code with documented Readme.

3 Evaluation

Each project is evaluated based on two components: an **instructor grade** and a **peer evaluation grade**. The final score combines both components to reward individual contribution within the team.

Instructor Grade

Each student receives an individual grade $G_i \in [0, 100]$ assigned by the instructors based on the technical report, code repository, and final presentation.

Peer Evaluation

Each team consists of n students, where $n \in \{3, 4\}$. Every team member evaluates the contribution of their teammates on a scale of 0 to 100, and also provides a self-assessment score. The peer score C_i for student i is computed as the average of the scores received from the **other** $n - 1$ **members only** — self-assessment is not included in this average:

$$C_i = \frac{1}{n-1} \sum_{\substack{j=1 \\ j \neq i}}^n s_{ji}$$

where s_{ji} is the score given by student j to student i .

Peer Adjustment Factor

The self-assessment score s_{ii} is used solely to compute the adjustment factor. It measures whether a student over- or under-estimated their own contribution relative to how their peers rated them:

$$\Delta_i = C_i - s_{ii}$$

A positive Δ_i means peers rated the student higher than their own self-assessment — indicating modesty and likely strong contribution. A negative Δ_i means the student overestimated their own contribution relative to their peers' assessment.

The adjustment factor is defined as:

$$F_i = \begin{cases} 1.67 & \text{if } \Delta_i \geq 20 \\ 1.33 & \text{if } 10 \leq \Delta_i < 20 \\ 1.00 & \text{if } -10 < \Delta_i < 10 \\ 0.50 & \text{if } -20 \leq \Delta_i \leq -10 \\ 0.00 & \text{if } \Delta_i < -20 \end{cases}$$

Final Score

The final individual score is computed as:

$$P_i = \min(120, 0.70 \cdot G_i + 0.30 \cdot G_i \cdot F_i)$$

The instructor grade accounts for 70% of the final score and cannot be overridden by peer evaluation. The remaining 30% is modulated by the peer adjustment factor F_i , allowing a maximum bonus of **+20 points** for students rated significantly higher by their peers, and a maximum deduction of **-30 points** for students who significantly overestimated their own contribution.

Example

Consider a team of $n = 4$ students with instructor grade $G_i = 100$ for all members, but different peer scores:

Student	G_i	C_i	Δ_i	F_i	P_i
Student A	100	100	+25	1.67	120
Student B	100	90	+15	1.33	109.9
Student C	100	85	0	1.00	100
Student D	100	60	-25	0.00	70

4 Project List

4.1 Project-1 Object Detection (2D)

In this project, you will develop and evaluate a 2D object detection system for autonomous driving using images from the KIITI Benchmark Object Detection. The goal of this task is to detect and localize objects in road scenes by predicting 2D bounding boxes around relevant objects and assigning them to the correct object class.

The KITTI object detection benchmark contains 7,481 training images and 7,518 test images, with a total of approximately 80,256 labeled objects. All images are color images stored in PNG format. The dataset includes annotations for common road participants such as cars, pedestrians, and cyclists. Your task is to design a model that takes an image as input and outputs bounding boxes and class labels for the objects present in the scene.

For evaluation, object detection performance is measured using precision–recall curves, following the criteria introduced in the PASCAL VOC Object Detection Benchmark. The main evaluation metric is Average Precision (AP), which summarizes the precision–recall curve into a single score. To consider a detection correct, the predicted bounding box must overlap sufficiently with the ground truth bounding box. In KITTI, this is measured using Intersection over Union (IoU).

The required IoU thresholds are:

- 70% overlap for cars
- 50% overlap for pedestrians and cyclists

KITTI also defines three difficulty levels based on object visibility and size:

- Easy: minimum bounding box height of 40 pixels, fully visible objects, and maximum truncation of 15%.
- Moderate: minimum bounding box height of 25 pixels, partly occluded objects, and maximum truncation of 30
- Hard: minimum bounding box height of 25 pixels, objects that are difficult to see, and maximum truncation of 50

For comparison between methods, performance is primarily evaluated on the moderate difficulty level, which represents typical real-world driving scenarios.

To complete this project, you are expected to perform the following tasks:

- Prepare the KITTI object detection dataset for training and evaluation. This includes downloading the dataset, parsing annotation files, and organizing the images and labels into a format suitable for your chosen deep learning framework.
- Implement or adapt a 2D object detection model capable of predicting bounding boxes and class labels for objects in road scenes. You may implement a model from scratch or adapt an existing architecture such as Faster R-CNN, YOLO, SSD, or another method proposed in the literature.
- Train the detection model using the KITTI training dataset. You should design an appropriate training pipeline including data preprocessing, augmentation, loss functions, and hyperparameter selection.
- Perform inference on validation images and generate predicted bounding boxes, confidence scores, and class labels for detected objects.
- Evaluate the detection performance using the KITTI evaluation protocol. Compute precision-recall curves and Average Precision (AP) for the object classes (Car, Pedestrian, Cyclist) using the required IoU thresholds.
- Analyze the performance of your model across the different difficulty levels (Easy, Moderate, Hard), and discuss which scenarios the model handles well and where it struggles.
- Provide qualitative visualizations of detection results by overlaying predicted bounding boxes on sample images. These visualizations should highlight both successful detections and typical failure cases.
- Write a short technical report summarizing the model architecture, training procedure, evaluation results, and insights gained from the experiments.

You may use modern deep learning object detection architectures, such as region-based detectors or single-shot detectors, or implement a method proposed in a relevant research paper. You are also allowed to use additional datasets for training if you believe they can help improve the performance or robustness of your model. However, the final evaluation of your method must be conducted using the KITTI benchmark protocol to ensure a fair and consistent comparison.

More details are here [KITTI Benchmark Object Detection](#)

4.2 Project-2 Visual Odometry / SLAM

In this project, you will develop a Visual Odometry (VO) or Simultaneous Localization and Mapping (SLAM) system using data from the KITTI Vision Benchmark Suite. The goal of this task is to estimate the motion of a vehicle over time by analyzing sequential sensor measurements and reconstructing the trajectory of the vehicle as it moves through the environment.

The KITTI odometry benchmark consists of 22 driving sequences recorded using sensors mounted on a vehicle. The sequences are provided in lossless PNG image format and include synchronized sensor data such as stereo camera images, calibration information, and optionally LiDAR measurements. Among these sequences, 11 sequences (00–10) include ground truth trajectories that can be used for training and validation, while 11 sequences (11–21) are used for evaluation and do not provide ground truth.

In this project, you may develop a system using monocular visual odometry, stereo visual odometry, LiDAR-based SLAM, or a method that combines visual and LiDAR information. The system should automatically estimate the motion of the vehicle and produce a trajectory describing the vehicle's position over time.

Your approach must satisfy two key conditions. First, the system must be fully automatic, meaning that no manual intervention (such as manual loop closure tagging or trajectory correction) is allowed. Second, the same parameter configuration must be used for all sequences, ensuring that the system generalizes across different driving scenarios.

The performance of your method will be evaluated by comparing the estimated trajectory with the ground truth trajectory provided in the dataset. The evaluation measures both translational error and rotational error across trajectory segments of different lengths. Specifically, the benchmark computes errors for subsequences with lengths ranging from 100 meters to 800 meters.

The evaluation metrics are defined as follows:

- Translational Error: measured as the average percentage error in estimated translation over a trajectory segment.
- Rotational Error: measured as the average rotational drift in degrees per meter traveled.

The final performance score is computed as the average translational and rotational error across all evaluated trajectory segments.

To complete this project, you are expected to perform the following tasks:

- Prepare the KITTI odometry dataset for experimentation. This includes downloading the sequences, reading calibration files, and organizing the sensor data (e.g., monocular images, stereo images, or LiDAR data) required by your method.
- Implement or adapt a Visual Odometry or SLAM pipeline that estimates the motion of the vehicle from sequential sensor measurements. The system may be monocular VO, stereo VO, LiDAR-based SLAM, or a multi-sensor approach.
- Implement the core components of the pipeline, such as feature detection and matching, motion estimation, pose estimation, and trajectory accumulation. Learning-based approaches may also be used if appropriate.

- Run the system on the KITTI training sequences (00–10) and generate estimated trajectories representing the vehicle’s pose over time.
- Evaluate the estimated trajectories by comparing them with the ground truth trajectories provided in the dataset. Compute translational and rotational errors according to the KITTI odometry benchmark protocol.
- Visualize the estimated trajectories together with the ground truth trajectories in order to qualitatively analyze the motion estimation performance.
- Investigate how the system behaves under different driving conditions such as turns, dynamic objects, or long trajectories, and discuss the main sources of error.
- Write a short technical report describing the system architecture, implementation details, evaluation results, and key observations from the experiments.

You may use classical geometric methods, modern learning-based approaches, or hybrid methods that combine both. You may also implement an approach proposed in a relevant research paper. External datasets or pretrained models may be used if they help improve performance, but the final evaluation must follow the KITTI odometry benchmark protocol.

More details are here [KITTI Visual Odometry / SLAM](#)

4.3 Project-3 Road and Lane Detection

In this project, you will develop a road and lane detection system for autonomous driving scenes using data from the KITTI Vision Benchmark Suite. The objective of this task is to accurately identify drivable road areas and, where available, detect the ego-lane (the lane in which the vehicle is currently driving). Your model should analyze camera images and produce pixel-level predictions that separate the road surface from the surrounding environment while also identifying lane structure when lane markings are present.

The KITTI road and lane estimation benchmark contains 289 training images and 290 test images collected in real-world urban driving environments. The dataset is divided into three categories of road scenes:

- uu – urban unmarked roads (98 training / 100 test images)
- um – urban roads with lane markings (95 training / 96 test images)
- umm – urban roads with multiple marked lanes (96 training / 94 test images)
- urban – combination of the three categories above

Ground truth annotations were generated through manual labeling of the images. Two types of terrain annotations are provided:

- **road**: the complete drivable road surface, representing the union of all lanes
- **lane**: the ego-lane, i.e., the lane the vehicle is currently driving in (available only for the *um* category)

Ground truth labels are provided only for the training images, while the test images are unlabeled and used for evaluation.

The dataset also provides additional sensor modalities that can support experimentation, including:

- Left color images, calibration files, and training labels
- Right color image extension
- Grayscale image extension
- Velodyne LiDAR point cloud extension
- OXTS GPS/IMU measurements
- Development kit for evaluation and benchmarking
- Mapping between training images and raw driving sequences

For this project, the primary input modality should be the camera images. However, additional sensor data may be used if it improves the performance of the proposed method.

Model performance is evaluated in the bird’s-eye-view (BEV) space, which projects the detected road region into a top-down representation relative to the vehicle. This representation enables consistent evaluation of road geometry and lane alignment.

The benchmark uses several pixel-level evaluation metrics commonly used in computer vision:

- MaxF: Maximum F1-measure across all decision thresholds
- AP: Average Precision
- PRE: Precision at the operating point corresponding to MaxF
- REC: Recall at the operating point corresponding to MaxF
- FPR: False Positive Rate
- FNR: False Negative Rate
- F1: Harmonic mean of precision and recall
- HR: Hit Rate

For this project, the primary evaluation metric will be the **Maximum F1-score (MaxF)**. The F1-score is defined as the harmonic mean of precision and recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

MaxF corresponds to the maximum value of this score across different decision thresholds.

To complete this project, you will design and implement a road and lane detection model capable of accurately identifying drivable road regions and estimating the ego-lane in urban driving scenes. Your work should include model development, training, prediction generation, quantitative evaluation, and analysis of the results.

To complete this project, you are expected to perform the following tasks:

- Prepare the KITTI road and lane dataset for experimentation, including downloading the data, parsing annotations, and organizing images and labels for training.
- Implement or adapt a road and lane detection model capable of producing pixel-level predictions for road regions and, when available, ego-lane segmentation.
- Train the model using the provided KITTI training images. The training pipeline should include appropriate preprocessing, data augmentation, and hyperparameter selection.
- Generate predictions for validation images, producing segmentation masks that identify drivable road areas and lane regions.
- Evaluate the performance of your model using the KITTI road benchmark protocol and compute the MaxF metric for the predicted results.
- Visualize qualitative results by overlaying predicted road and lane regions on sample images to illustrate both successful detections and typical failure cases.
- Analyze the strengths and limitations of your method, including how it performs under different scene types such as unmarked roads, marked lanes, and multi-lane scenarios.
- Write a short technical report describing the model architecture, implementation details, experimental results, and insights gained from the evaluation.

You may implement modern deep learning architectures for segmentation or reproduce a method proposed in a relevant research paper. Additional datasets may be used for training if they improve performance, but the final evaluation must follow the KITTI road and lane estimation benchmark protocol.

More details are here [KITTI Benchmark Road/Lane Detection](#)

4.4 Project-4 [Segmenting and Tracking Every Pixel](#)

In this project, you will develop a spatio-temporal perception system for autonomous driving using data from the KITTI Vision Benchmark Suite. The objective of this task is to jointly perform dense semantic segmentation and object tracking across video frames. Your system must assign a semantic label to every pixel in each frame while maintaining consistent identities for tracked objects as they move through the scene.

This task is based on the *Segmenting and Tracking Every Pixel (STEP)* benchmark, which extends the KITTI tracking benchmark by providing dense pixel-level annotations for every frame in a video sequence. The dataset contains 21 training sequences and 29 test sequences recorded in real urban driving environments. Each sequence consists of consecutive frames captured by a vehicle-mounted camera, forming a video stream that represents dynamic traffic scenes.

In this benchmark, every pixel in every frame is assigned a semantic class label representing the scene category, such as road, building, vegetation, sky, vehicle, or pedestrian. In addition to semantic segmentation labels, the dataset provides unique tracking identifiers for pixels belonging to the most important dynamic object classes:

- Car

- Pedestrian

All pixels corresponding to the same physical object instance across frames share the same tracking ID. This allows evaluation of whether the system can consistently track objects over time. Therefore, the model must not only segment objects correctly in each frame but also maintain temporal consistency by preserving object identities throughout the sequence.

Performance for the STEP task is evaluated using the [Segmentation and Tracking Quality \(STQ\)](#) metric. This metric jointly evaluates segmentation accuracy and tracking consistency by combining two components:

- **Segmentation Quality (SQ)**: measures the pixel-level segmentation accuracy using mean Intersection over Union (IoU).
- **Association Quality (AQ)**: measures how well object identities are preserved across frames.

To complete this project, you will design and implement a video-based perception model capable of producing pixel-level semantic predictions and temporally consistent object tracks in complex urban scenes. Your work should include model design, training, inference across video sequences, quantitative evaluation, and analysis of both segmentation and tracking performance.

To complete this project, you are expected to perform the following tasks:

- Prepare the KITTI STEP dataset for experimentation, including downloading the sequences, parsing the annotation format, and organizing frames and labels for training and evaluation.
- Implement or adapt a segmentation and tracking pipeline capable of producing pixel-level semantic segmentation and maintaining consistent object identities across frames.
- Train the model using the provided training sequences. Your training pipeline should include appropriate preprocessing, temporal data handling, and hyperparameter selection.
- Run inference on video sequences and generate predictions that include semantic segmentation maps and tracking IDs for objects belonging to the car and pedestrian classes.
- Evaluate the performance of your method using the STEP benchmark protocol and compute the Segmentation and Tracking Quality (STQ) metric.
- Visualize qualitative results by displaying segmentation maps and tracked object identities across consecutive frames to illustrate temporal consistency.
- Analyze the strengths and limitations of your approach, including challenges such as object occlusions, appearance changes, and complex scene dynamics.
- Write a short technical report describing the system architecture, implementation details, evaluation results, and key insights from your experiments.

You may implement modern architectures such as video segmentation networks, tracking-by-detection pipelines, or joint segmentation-tracking models proposed in recent research literature. Additional datasets or pretrained models may be used if they improve performance, but the final evaluation must follow the KITTI STEP benchmark protocol and the STQ evaluation metric.

More details are here [KITTI Segmenting and Tracking Every Pixel](#)